

Note Completo dai 4 File C#

VALUE TYPE vs REFERENCE TYPE

Value Types (int, double, bool, struct)

Comportamento: "Copia il valore"

```
int fa = 10;
int fb = fa; // Copia il valore
fa = -5;

Console.WriteLine(fa); // -5
Console.WriteLine(fb); // 10 (rimane 10!)
```

Memory Layout:

```
STACK

fa: 10      fb: 10
"          "

[10]       [10]
(separate, indipendenti)

Dopo fa = -5:
fa: -5      fb: 10
"          "

[-5]       [10]
(Non si influenzano)
```

Conclusione: Ogni variabile ha una **copia indipendente** del valore.

Reference Types (array, string, classi)

Comportamento: "Copia il riferimento"

```
int[] a = new int[5] { 1, 2, 3, 4, 5 };
int[] b = a; // Copia il RIFERIMENTO (non l'array!)

b[3] = -100; // Modifica l'array attraverso b
```

```
Console.WriteLine(a[3]); // -100 (cambiato!)
Console.WriteLine(b[3]); // -100
```

Memory Layout:

STACK	HEAP
a	[1, 2, 3, 4, 5]
0xA001 \$	
b	(Stesso indirizzo!)
Dopo b[3] = -100:	
a	[1, 2, 3, -100, 5]
0xA001 \$	
b	(Entrambi vedono la modifica!)

Conclusione: Entrambi **puntano allo stesso array in memoria**. Modifiche visibili da entrambe le variabili!

ATTENZIONE: Array.Length

```
int[] a = new int[5] { 1, 2, 3, 4, 5 };
int[] b = a;

Console.WriteLine(a.Length); // 5
Console.WriteLine(b.Length); // 5 (stesso array!)

// Non potrai mai avere array con lunghezze diverse se assegnati l'uno
all'altro
```

Array - Operazioni Base

Dichiarazione e Inizializzazione

```
// Sintassi classica
int[] a = new int[5] { 1, 2, 3, 4, 5 };

// Con inferenza di tipo
var a = new int[] { 1, 2, 3, 4, 5 };
```

```
// Array vuoto
int[] empty = new int[5]; // { 0, 0, 0, 0, 0 }
```

Iterazione

```
int[] a = new int[5] { 1, 2, 3, 4, 5 };

// Con for
for (int i = 0; i < a.Length; i++) {
    Console.Write(a[i] + " ");
}

// Con foreach (più moderno)
foreach (int num in a) {
    Console.Write(num + " ");
}
```

Proprietà

```
int[] a = new int[5] { 1, 2, 3, 4, 5 };
a.Length; // 5 (numero elementi)
```

ESERCIZIO 1: Guess Number with Bits (Binary Decomposition)

Concetto: Ogni numero 1-128 può essere scomposto in potenze di 2!

```
int GuessNumberWithBits() {
    Console.WriteLine("Think of a number between 1 and 128");
    int guessedNumber = 0;
    int[] powers = { 1, 2, 4, 8, 16, 32, 64, 128 };

    for (int i = 0; i < 8; i++) {
        Console.Write($"Does the number contain {powers[i]}? (y/n): ");
        string answer = Console.ReadLine().ToLower();

        if (answer == "y" || answer == "yes") {
            guessedNumber += powers[i];
        }
    }
}
```

```
    return guessedNumber;
}
```

Come Funziona

Esempio: Numero = 87

$87 = 64 + 16 + 4 + 2 + 1$

Domande:

- Contiene 1? SÌ ' 1
- Contiene 2? SÌ ' 2
- Contiene 4? SÌ ' 4
- Contiene 8? NO ' (skip)
- Contiene 16? SÌ ' 16
- Contiene 32? NO ' (skip)
- Contiene 64? SÌ ' 64
- Contiene 128? NO ' (skip)

Somma: $1 + 2 + 4 + 16 + 64 = 87$

Logica Binaria

Ogni numero è somma di potenze di 2:

$128 = 2^7$

$64 = 2^6$

$32 = 2^5$

$16 = 2^4$

$8 = 2^3$

$4 = 2^2$

$2 = 2^1$

$1 = 2^0$

Massimo 8 domande per indovinare qualsiasi numero 1-128!

ESERCIZIO 2: Guessing Game con Modalità

```
var mode = GetMode();
int max = GetMax(mode);
int secret = new Random().Next(1, max + 1);
```

```
Console.WriteLine($"I've thought of a number between 1 and {max}.");
```

```

Console.WriteLine("Try to guess it");

for (int tries = 1; ; tries++) {
    int guess = ReadInt("> ");
    var result = CheckGuess(secret, guess);

    if (result == GResult.Exact) {
        Console.WriteLine($"You guessed in {tries} tries.");
        break; // •Esce dal loop quando indovina
    }

    Console.WriteLine($"Your guess is too {result}.");
}

```

Flusso del Programma

1. GetMode() ' Chiede Easy/Medium/Hard
2. GetMax(mode) ' Setta range (10/100/1000)
3. new Random().Next() ' Genera numero segreto
4. FOR LOOP
 - ReadInt() ' Legge indovina da utente
 - CheckGuess() ' Confronta (Low/Exact/High)
 - IF Exact ' break ' Esce dal loop
 - ELSE ' Continua ' Prossimo tentativo (tries++)

FUNZIONI SUPPORTO

1. CheckGuess() - Confronta i Numeri

```

GResult CheckGuess(int secret, int guess) {
    if (guess == secret) return GResult.Exact;
    if (guess < secret) return GResult.Low;
    return GResult.High;
}

```

Condizione	Ritorno	Significato
<code>guess == secret</code>	Exact	Indovinato!
<code>guess < secret</code>	Low	Il numero è più alto
<code>guess > secret</code>	High	Il numero è più basso

2. ReadInt() - Legge Intero Validato

```
int ReadInt(string prompt) {  
    for (;;) {  
        Console.Write(prompt);  
        if (int.TryParse(Console.ReadLine(), out int result))  
            return result;  
    }  
}
```

Logica:

- Loop infinito finché non ricevi un numero valido
- `TryParse()` ritorna true se conversione riuscita
- Ritorna il numero convertito

3. GetMax() - Setta Range

Con Switch-Case (Moderno):

```
int GetMax(Mode mode) {  
    switch (mode) {  
        case Mode.Easy: return 10;  
        case Mode.Medium: return 100;  
        default: return 1000;  
    }  
}
```

Con If-Statement (Alternativo):

```
int GetMax(Mode mode) {  
    if (mode == Mode.Easy) return 10;  
    if (mode == Mode.Medium) return 100;  
    return 1000;  
}
```

Modalità	Range
Easy	1-10

Medium	1-100
Hard	1-1000

4. GetMode() - Scegli Modalità

```
Mode GetMode() {
    Console.WriteLine("Select a mode (E)asy, (M)edium, (H)ard: ");
    for (;;) {
        var key = Console.ReadKey(true).Key; // true = non stampa il
        // tast
        switch (key) {
            case ConsoleKey.E: return Mode.Easy;
            case ConsoleKey.M: return Mode.Medium;
            case ConsoleKey.H: return Mode.Hard;
            default:
                Console.WriteLine("Incorrect key");
                break;
        }
    }
}
```

Novità:

- `Console.ReadKey(true)` legge UN tasto SENZA stamparlo
- `.Key` estrae il tasto dal `ConsoleKeyInfo`
- Switch sulla proprietà `.Key`

ENUM - Tipi Enumerati

Cos'è un Enum?

Un enum definisce un **nuovo tipo** con un insieme fisso di valori.

```
enum Mode { Easy, Medium, Hard };
enum GResult { Exact, Low, High };
```

Vantaggi

Type-Safe: Il compilatore controlla i valori

Leggibile: `Mode.Easy` invece di `0`

Maintainable: Cambi in un unico posto

Intellisense: Auto-completamento IDE

Uso

```
Mode gameMode = Mode.Easy;

if (gameMode == Mode.Easy) {
    // ...
}

switch (gameMode) {
    case Mode.Easy: /* ... */ break;
    case Mode.Medium: /* ... */ break;
    case Mode.Hard: /* ... */ break;
}
```

SWITCH STATEMENT

Sintassi Base

```
switch (variabile) {
    case valore1:
        // Codice se variabile == valore1
        break; // IMPORTANTE! Esce dal switch
    case valore2:
        // Codice se variabile == valore2
        break;
    default:
        // Se nessun case corrisponde
        break;
}
```

Esempio

```
int choice = 2;

switch (choice) {
    case 1:
        Console.WriteLine("Scelta 1");
        break;
    case 2:
```



```
Console.WriteLine("Scelta 2");
break;
default:
Console.WriteLine("Scelta non valida");
break;
}
// Output: Scelta 2
```

BREAK Statement

Cosa Fa

Esce immediatamente dal loop o switch.

```
// Nel FOR loop
for (int i = 0; i < 10; i++) {
    if (i == 5) break; // Esce quando i == 5
    Console.WriteLine(i); // 0, 1, 2, 3, 4
}

// Nel WHILE loop
while (true) {
    if (condizione) break; // Esce dal loop infinito
}

// Nel SWITCH
switch (mode) {
    case Mode.Easy:
    // ...
    break; // Esce dal switch
}
```

CONSOLE.READKEY()

```
var key = Console.ReadKey(true); // true = non stampa
// oppure
var key = Console.ReadKey(false); // false = stampa (default)

key.Key; // Il tasto premuto (ConsoleKey.A, etc.)
key.Char; // Il carattere
key.Modifiers; // Shift, Ctrl, Alt premuti
```

Esempio

```
Console.WriteLine("Press a key...");
var key = Console.ReadKey(true);

if (key.Key == ConsoleKey.Enter) {
    Console.WriteLine("You pressed Enter!");
}
```

FLUSSO COMPLETO: Guessing Game

```
START
“
GetMode() • Legge E/M/H (con ReadKey)
“
GetMax() • Ritorna 10/100/1000
“
new Random().Next(1, max+1) • Genera numero segreto
“
FOR loop (tries=1; ; tries++)
    ReadInt() • Legge indovina validato
    CheckGuess() • Confronta (Low/Exact/High)
    IF Exact
        Print tries
        BREAK (esce loop)
    ELSE
        Print feedback + continua
“
END
```

BEST PRACTICES

1. Usa **Enum** per insiemi fissi di valori
2. Usa **Switch** invece di if-else multipli
3. **Break** in ogni case dello switch
4. **Valida sempre** l'input (TryParse, ReadKey)
5. Usa **Reference Types** quando modifiche serve ai tutti
6. Ricorda: Array sono **reference type** ' modifiche condivise
7. Usa **foreach** quando non serve l'indice
8. `Console.ReadKey(true)` per leggere tasti senza eco

RIASSUNTO CONCETTI CHIAVE

Concetto	Spiegazione
Value Type	Copia il valore (int, double, bool)
Reference Type	Copia il riferimento (array, string, classe)
Array	Collezione di elementi dello stesso tipo
Enum	Tipo personalizzato con valori fissi
Switch	Selezione multipla (migliore di if-else)
Break	Esce da loop o switch
TryParse	Converte string a numero (sicuro)
ReadKey	Legge un tasto dalla tastiera
Binary Decomposition	Scompone numero in potenze di 2

ESERCIZIO FINALE

Combina tutto:

- Usa **Enum** per modalità
- Usa **Switch** per sceglierne una
- Usa **Array** per i dati
- Usa **TryParse** per input validato
- Usa **Break** per uscire dal loop
- Comprendi **Value vs Reference Types**

Risultato: Un **Guessing Game** completo e robusto!